



APRENDIZAJE DE MÁQUINA

MÉTODOS DE ENSAMBLE

Especialización en Inteligencia Artificial
FIUBA

Dr. Ing. Facundo Adrián Lucianna
Esp. Lic. María Carina Roldán

En la clase anterior vimos ...

Árboles de decisión. Árboles de clasificación y árboles de regresión.

ÁRBOLES DE DECISIÓN

Los árboles de clasificación y regresión, conocidos como **CART** (*Classification and Regression Trees*), son una técnica de aprendizaje automático para resolver problemas tanto de **clasificación** como de **regresión**.

Los CART representan el proceso de decisión mediante una estructura en forma de árbol.

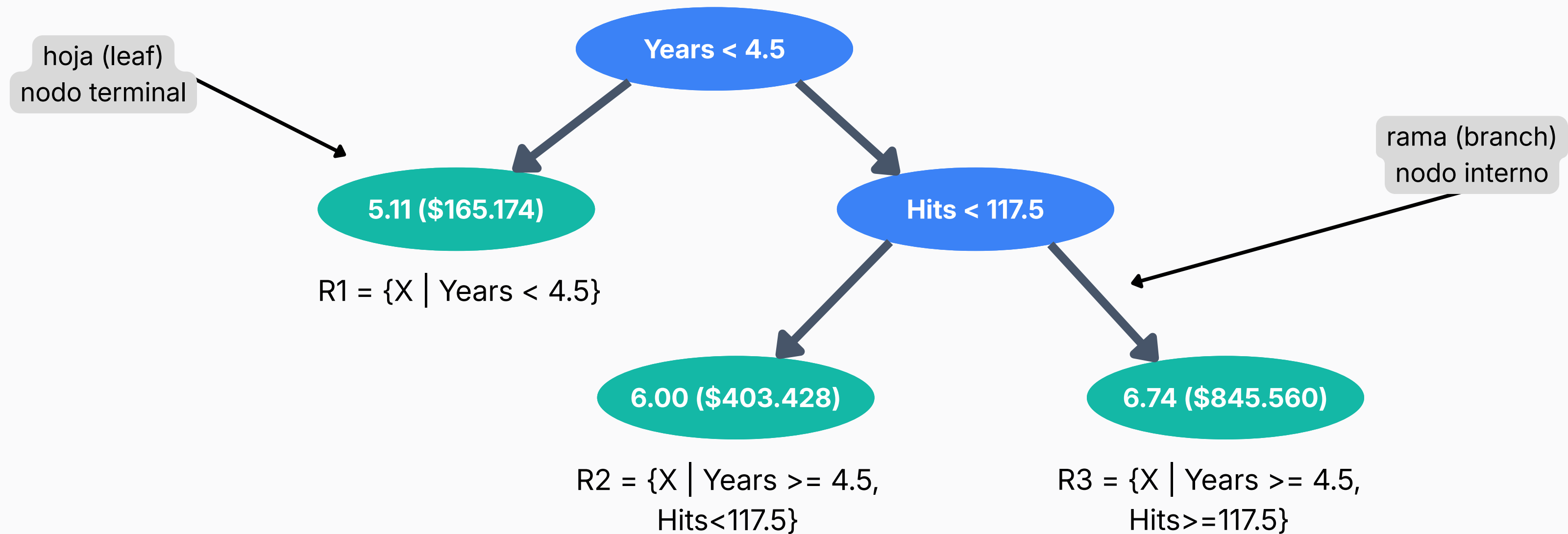


ÁRBOLES DE
CLASIFICACIÓN

ÁRBOLES DE
REGRESIÓN

ÁRBOLES DE REGRESIÓN

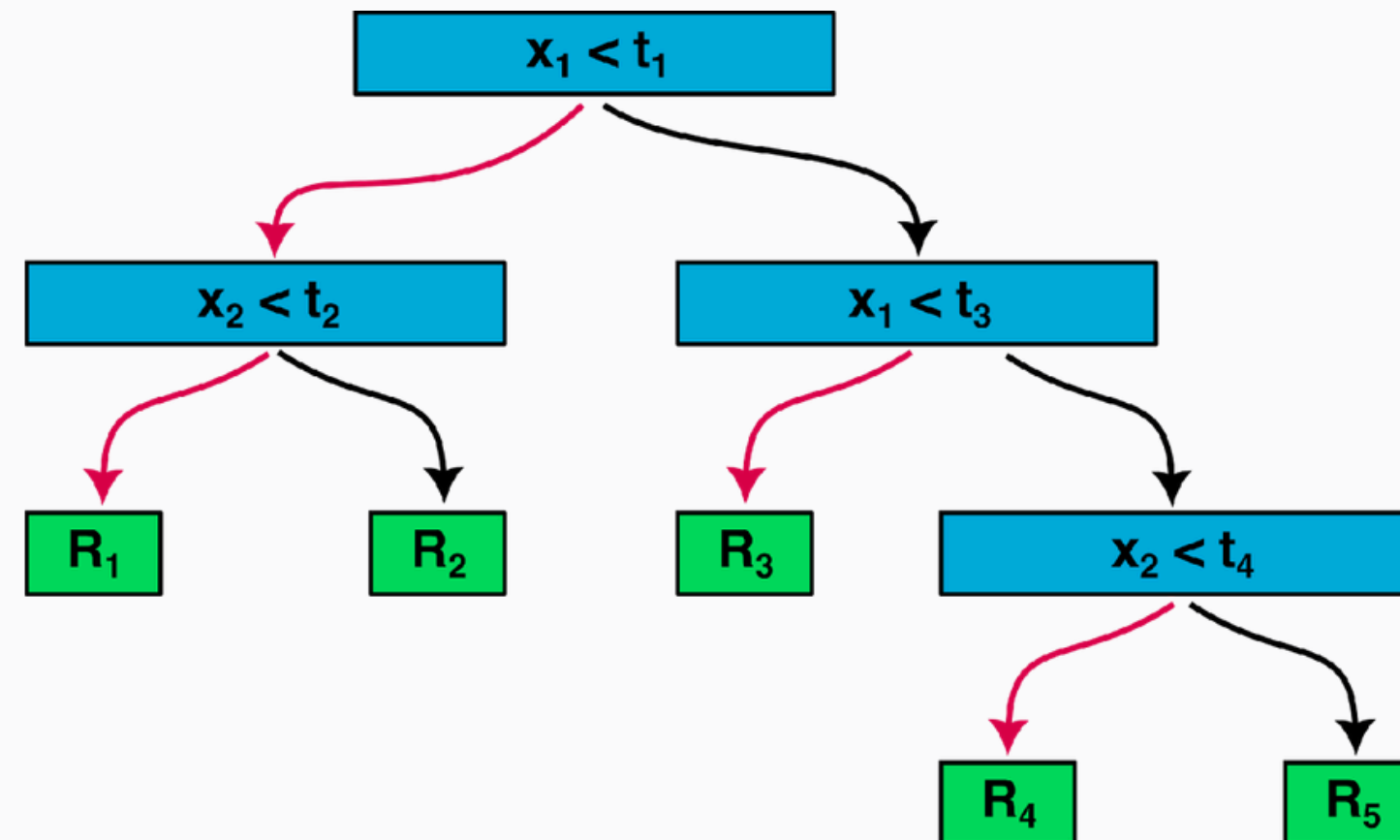
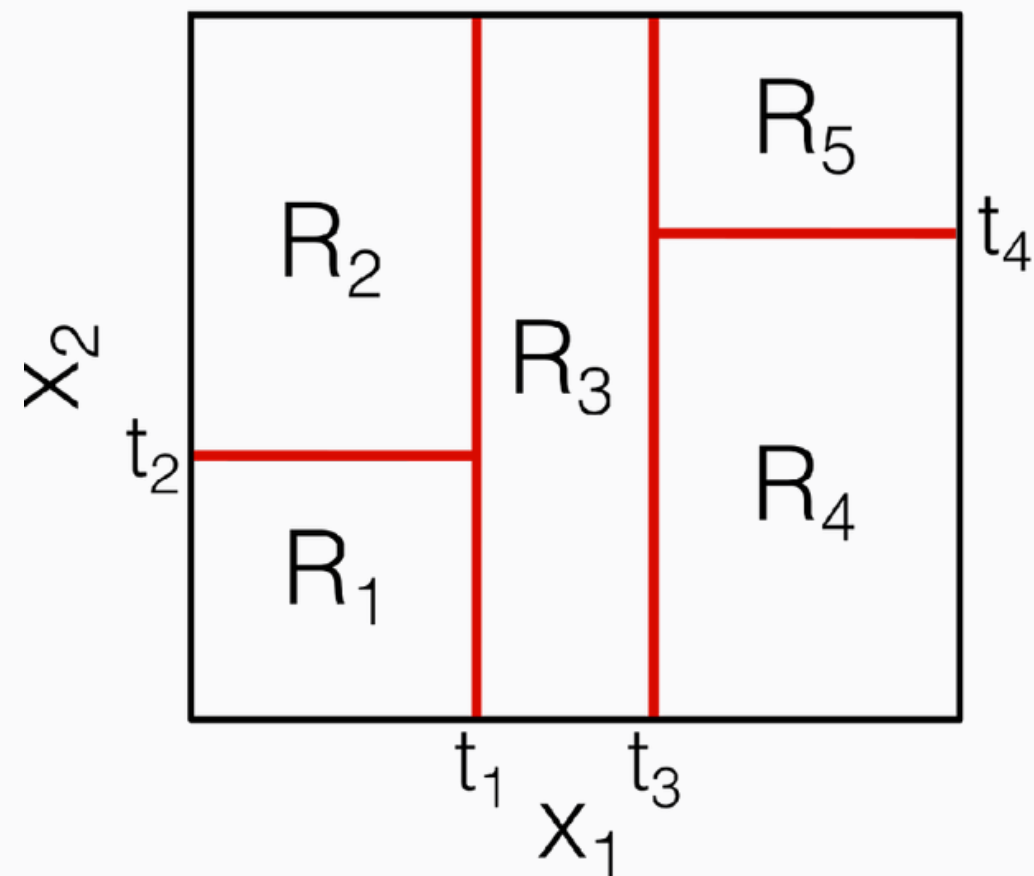
Los árboles CART permiten realizar predicciones basadas en reglas lógicas sencillas y fáciles de interpretar.



ÁRBOLES DE REGRESIÓN

Recursive binary splitting

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$



ÁRBOLES DE REGRESIÓN

Poda de árboles

En la fórmula de RSS introducimos un valor de penalización α .

Dado un valor de α , existe un subárbol T perteneciente a T_0 tal que:

$$RSS = \sum_{m=1}^L \sum_{i \in R_m} (y_i - y_{R_m})^2 + \alpha L$$

$\alpha \geq 0$
L número de hojas
del subárbol

RSS es mínimo.

α presenta un trade-off entre complejidad del árbol y su capacidad de ajustar a los datos (regularización).

- Si $\alpha=0$, el árbol elegido es T_0 .
- Cuanto más grande es α , el precio a pagar por el tamaño del árbol es mayor.

ÁRBOLES DE CLASIFICACIÓN

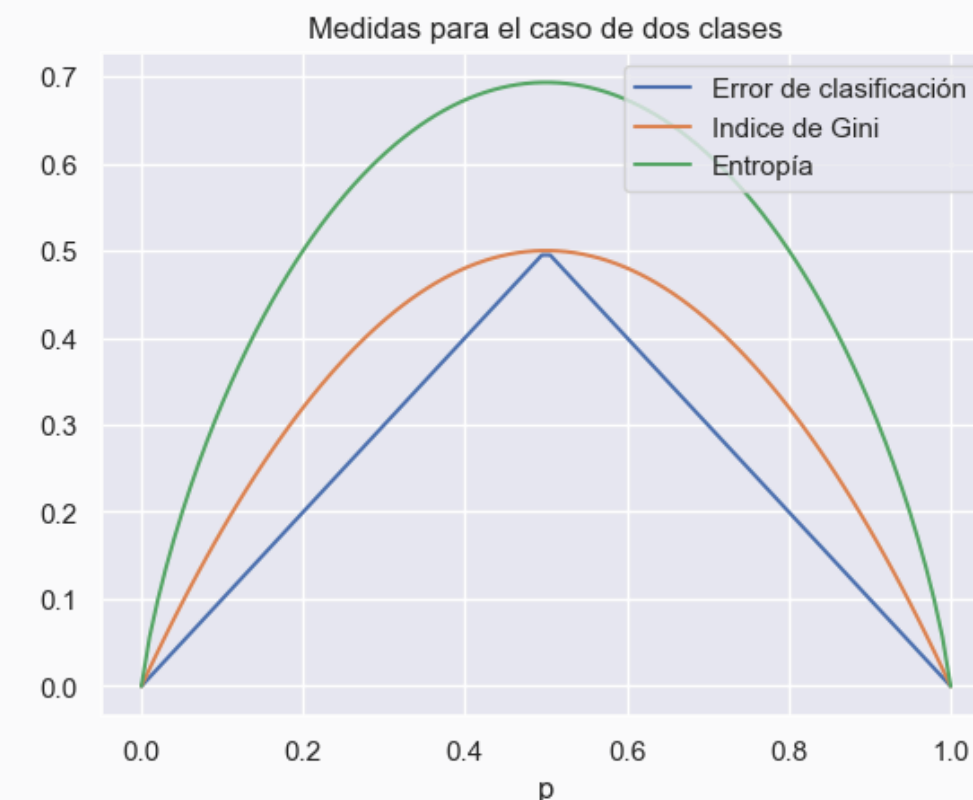
Un **árbol de clasificación** se usa para predecir una **variable cualitativa**.

La predicción se hace en base a la **clase que más ocurre** en las muestras de la hoja.

Al interpretar los resultados de un **árbol de clasificación**, además de la predicción nos interesa conocer las **proporciones de clase entre las observaciones de entrenamiento** que caen en cada región.

Por eso vimos dos **medidas de impureza**:

- el índice de Gini
- la entropía



MÉTODOS DE ENSAMBLE

MÉTODOS DE ENSAMBLE

¿Qué son?

En 1906, el estadístico Sir Francis Galton visitó una feria rural en Inglaterra donde se celebraba un concurso para adivinar el peso de un buey en exhibición. Se realizaron 800 estimaciones y, aunque las respuestas individuales variaron enormemente, tanto la media como la mediana de todas ellas resultaron estar a menos del 1% del peso real del animal.

Wisdom of the crowd
(Sabiduría de la
← multitud)

Si trasladamos esta idea a Machine Learning, observamos que al promediar la salida de un conjunto de predictores es muy probable obtener un mejor rendimiento que al considerar la salida de uno solo.

Dado que a un grupo de predictores se le conoce como **ensamble**, a esta técnica se le denomina **Métodos de Ensamble**.

MÉTODOS DE ENSAMBLE

Hagamos un experimento... Adivinen el peso de esta calabaza



Formulario: <https://forms.gle/HdX9RS4uXh77Xu6z7>

MÉTODOS DE ENSAMBLE

Por ejemplo, podríamos entrenar un grupo de árboles de decisión, utilizando para cada uno un subconjunto diferente de los datos de entrenamiento. Luego, generamos predicciones con cada árbol individual y determinamos la clase final mediante una votación mayoritaria.

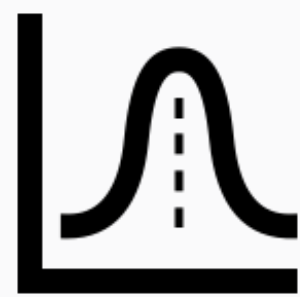
¿Cuándo usarlos?

Es frecuente utilizar métodos de ensamble cuando las arquitecturas simples (conocidas como *weak learners*) no logran el rendimiento esperado.

Para mejorar las métricas, combinamos estos predictores en un único modelo robusto.

***MODELOS QUE
VOTAN***

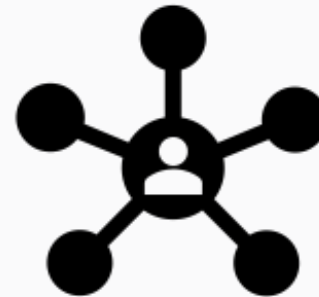
MODELOS QUE VOTAN



regresión
logística



SVC

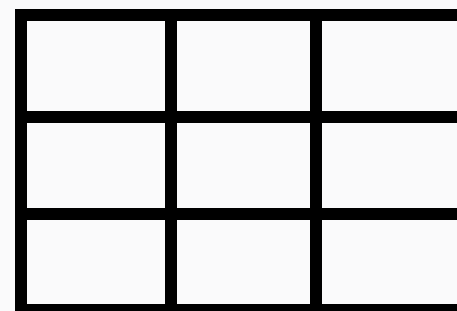


red neuronal



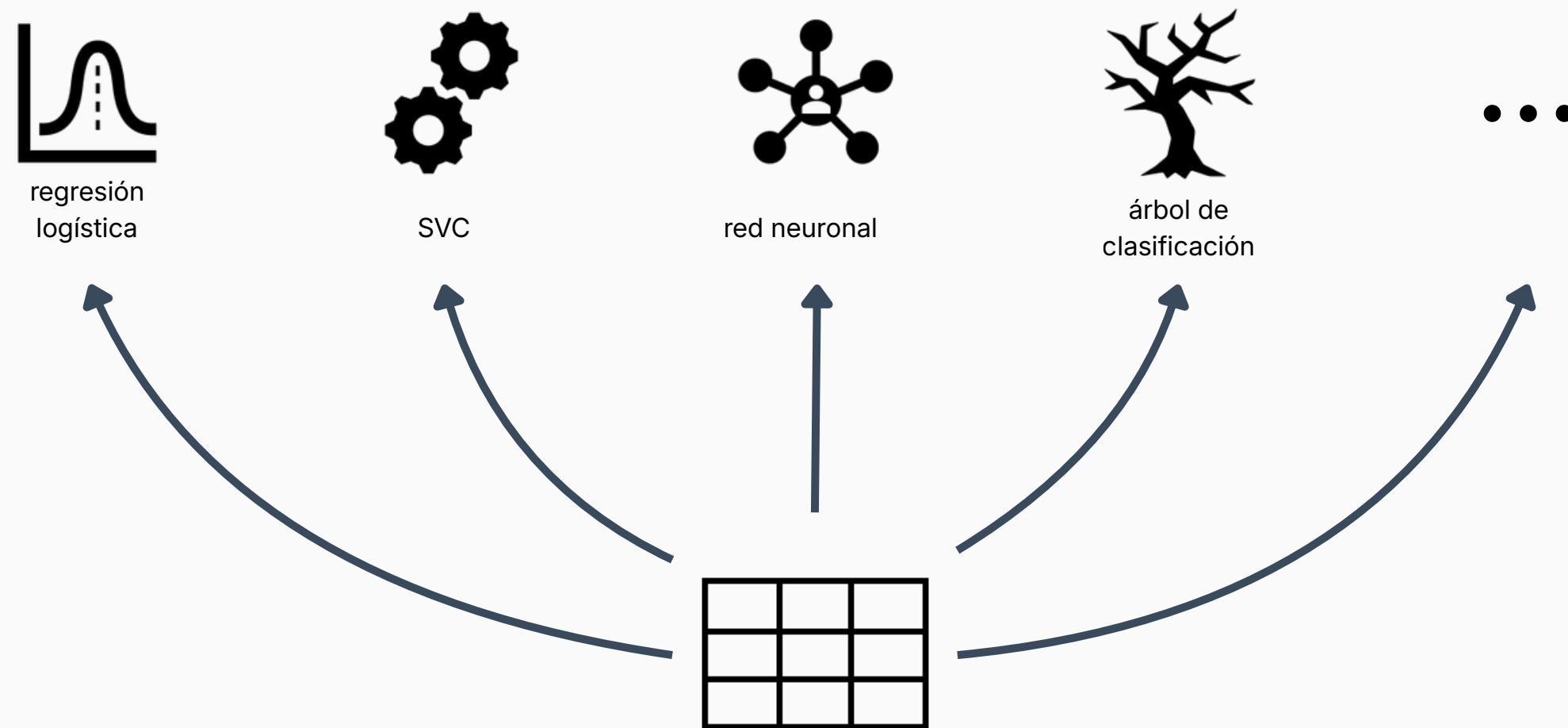
árbol de
clasificación

...



DATASET

MODELOS QUE VOTAN



Los métodos de ensamble funcionan mejor cuanto mayor sea la independencia entre los predictores.

Esto se puede lograr, por ejemplo, utilizando arquitecturas distintas para cada modelo.

MODELOS QUE VOTAN

Se entrena un conjunto de modelos. Luego la predicción final depende de la estrategia de votación:

Estrategia **HARD VOTING**

- Se basa en la moda de las predicciones.

Estrategia **SOFT VOTING**

- Se basa en promedios de predicciones. **Ventaja:** le da mayor peso a los modelos que tienen más confianza en su decisión.
- Se puede aplicar cuando los clasificadores pueden estimar probabilidades de clase (por ejemplo: tienen el método `predict_proba`).

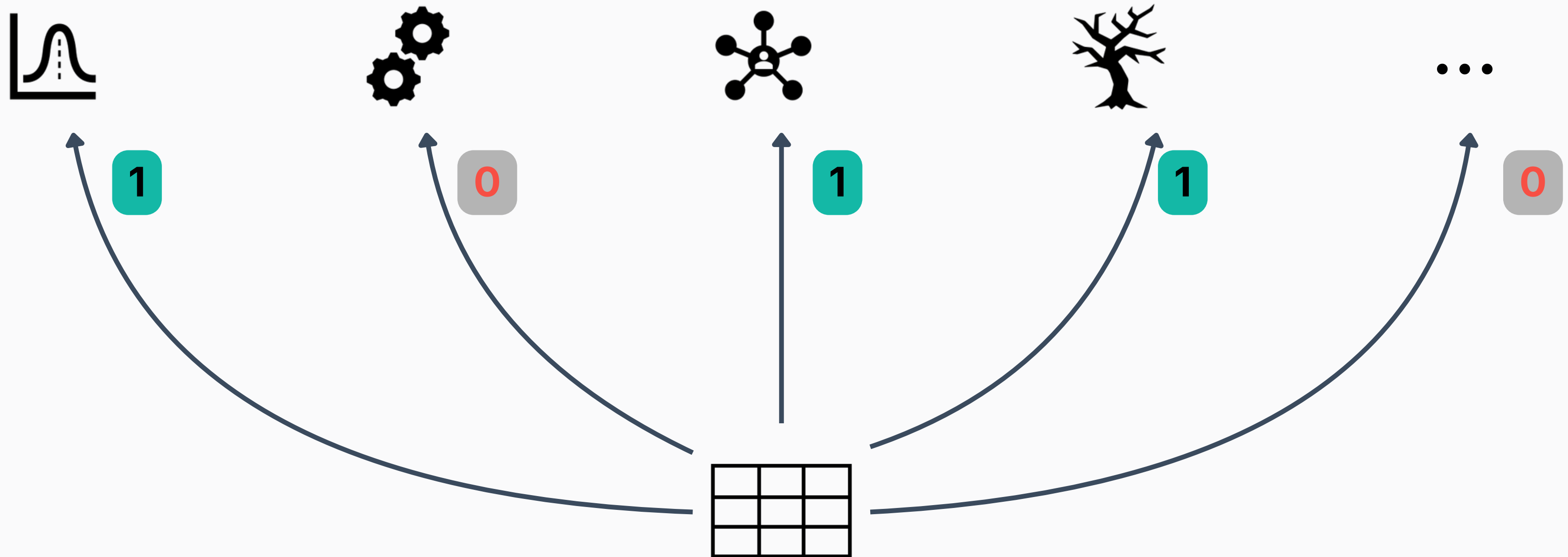
Estrategia **VOTING REGRESSOR**

- Se basa en promedios de predicciones (problemas de regresión).

MODELOS QUE VOTAN

HARD VOTING CLASSIFIERS

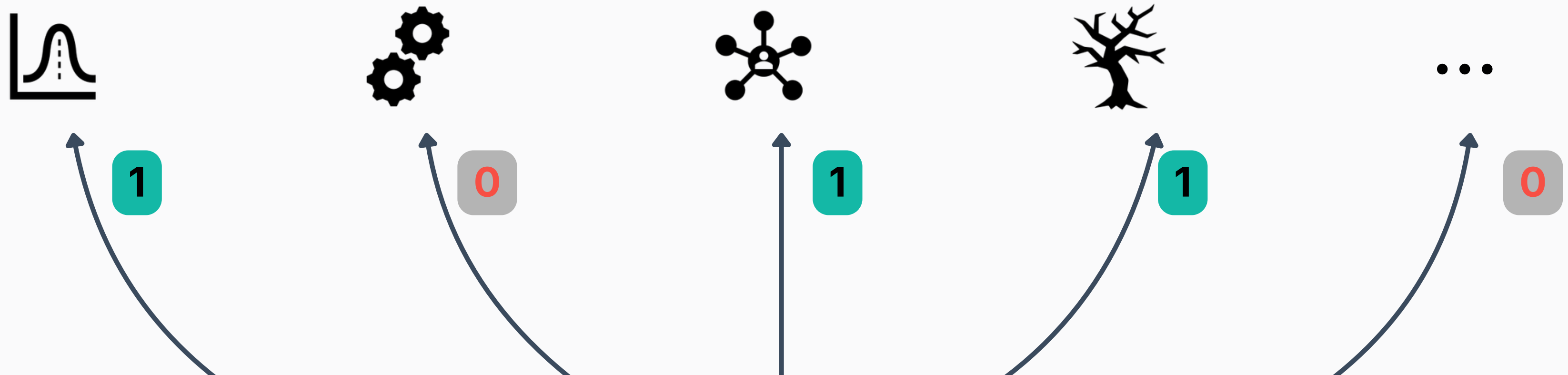
La clase predicha es la que recibe la mayoría de los votos por parte de los estimadores individuales.



MODELOS QUE VOTAN

HARD VOTING CLASSIFIERS

La clase predicha es la que recibe la mayoría de los votos por parte de los estimadores individuales.



1 es la moda. 3 modelos tienen como salida la clase 1.

0 2 modelos tienen como salida la clase 0.

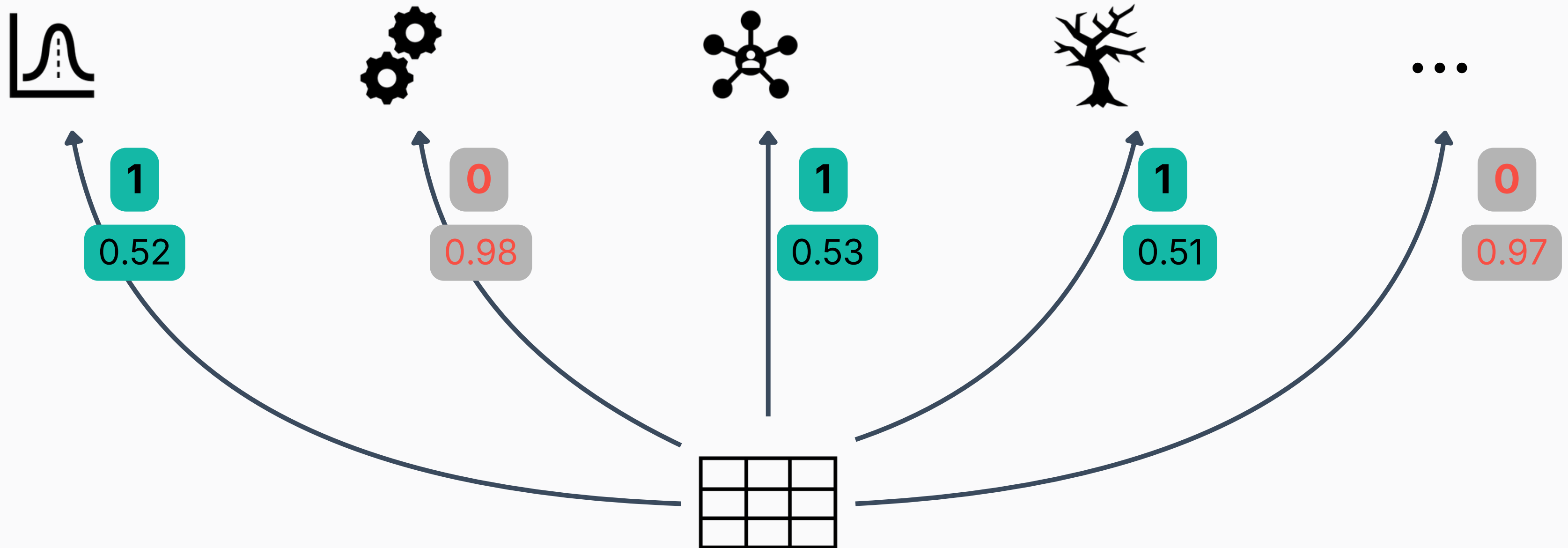
PREDICCIÓN
DEL ENSAMBLE:

1

MODELOS QUE VOTAN

SOFT VOTING CLASSIFIERS

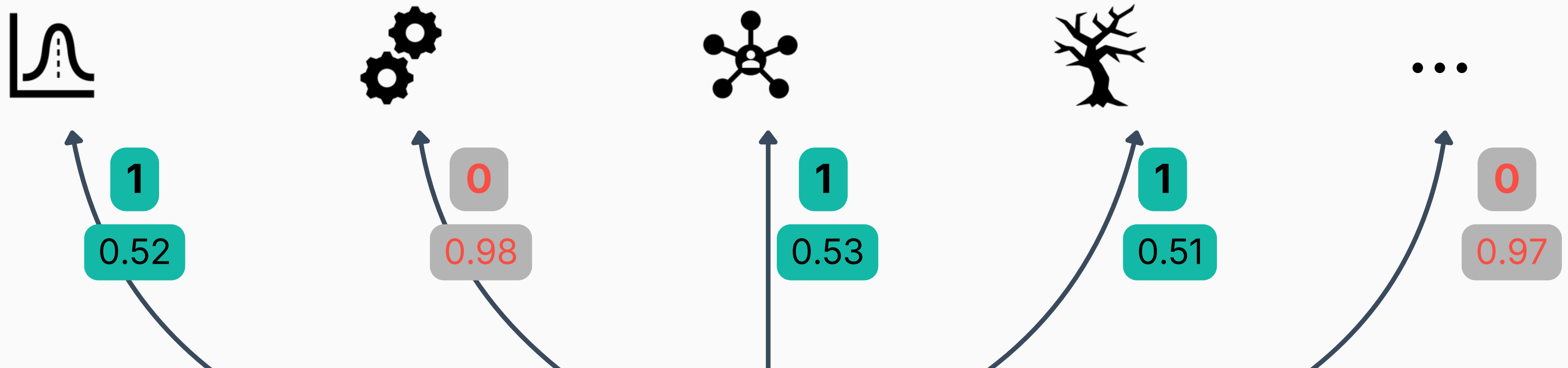
- Para cada clase, se promedian las probabilidades a través de todos los clasificadores individuales.
- se selecciona la clase con el promedio más alto (argmax).



MODELOS QUE VOTAN

SOFT VOTING CLASSIFIERS

- Para cada clase, se promedian las probabilidades a través de todos los clasificadores individuales.
- se selecciona la clase con el promedio más alto (argmax).



1 $(0.52 + 0.02 + 0.53 + 0.51 + 0.03) / 5 = \mathbf{0.312}$

0 $(0.48 + 0.98 + 0.47 + 0.49 + 0.97) / 5 = \mathbf{0.678}$

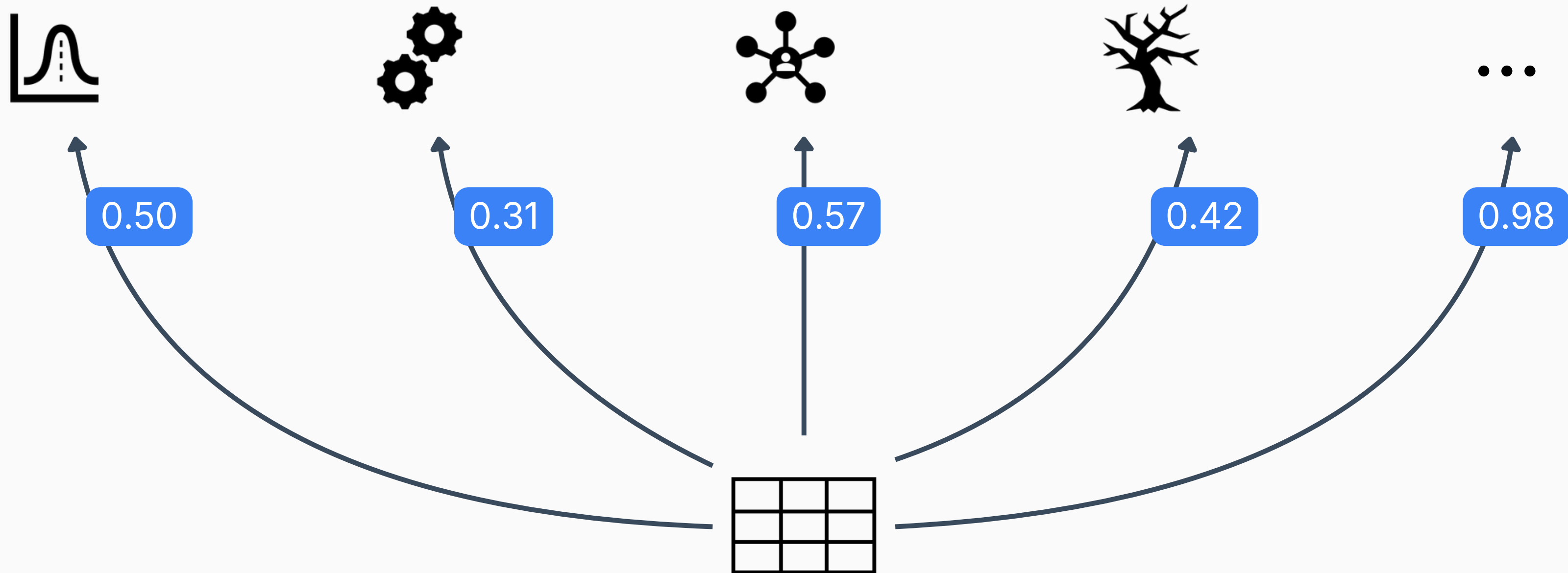
PREDICCIÓN
DEL ENSAMBLE:

0

MODELOS QUE VOTAN

VOTING REGRESSION

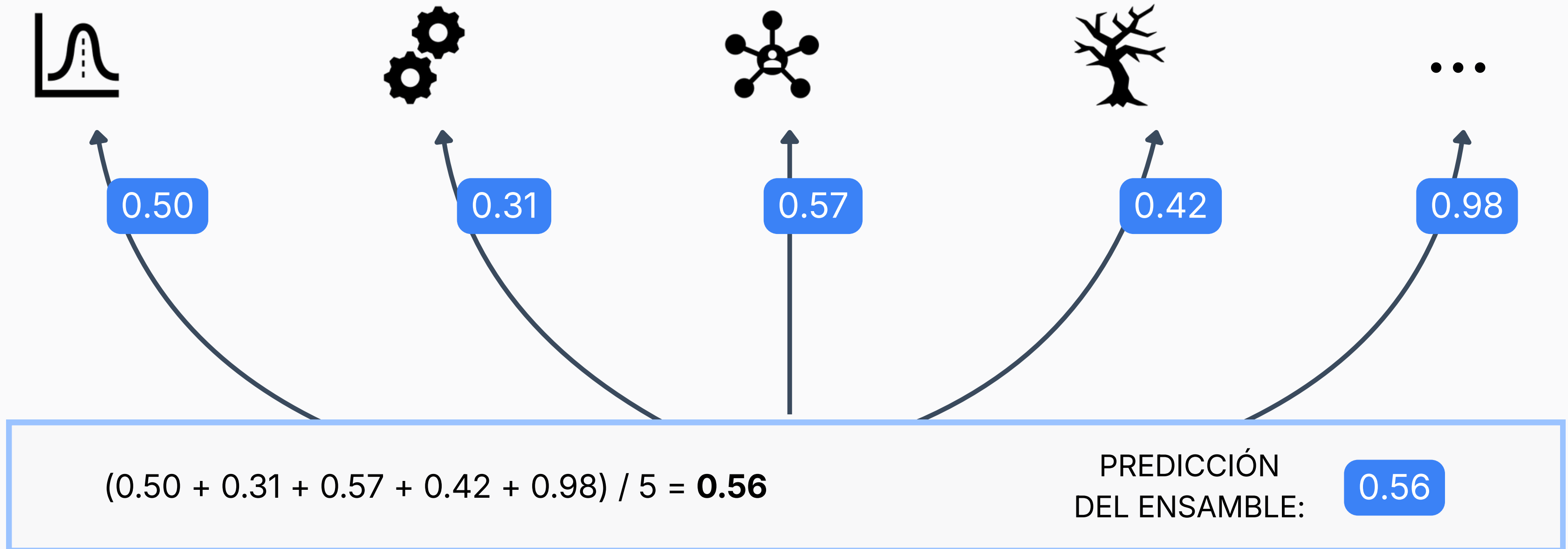
Se promedian las probabilidades de todos los modelos individuales.



MODELOS QUE VOTAN

VOTING REGRESSION

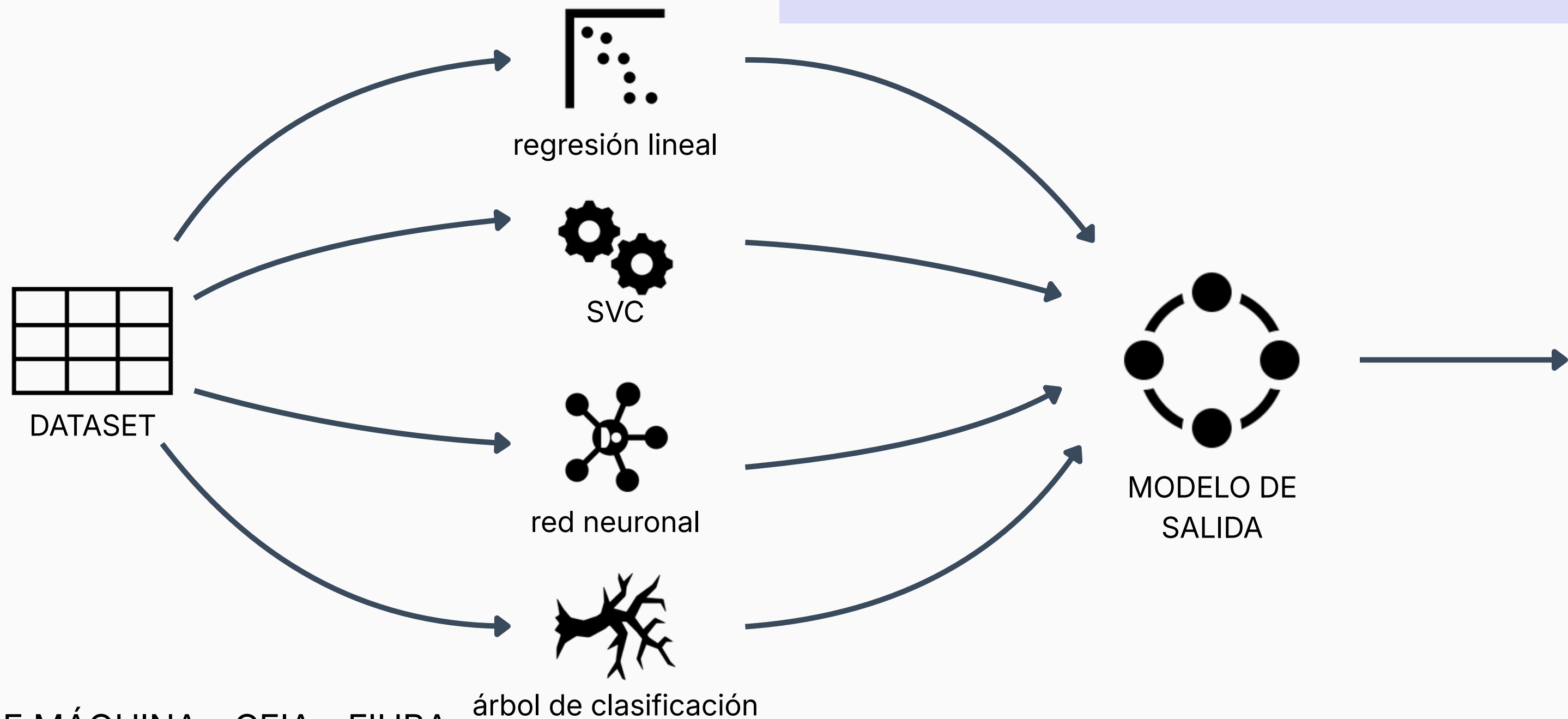
Se promedian las probabilidades de todos los modelos individuales.



MODELOS QUE VOTAN

STACKING (APILAMIENTO)

Una estrategia más sofisticada es entrenar un meta-modelo (o modelo final) que, para realizar la predicción definitiva, aprende a combinar las salidas de los predictores bases.





Vamos a practicar un
poco ...

BAGGING
(bootstrap aggregating)

BAGGING

¿Cómo logramos diversidad en nuestros predictores?

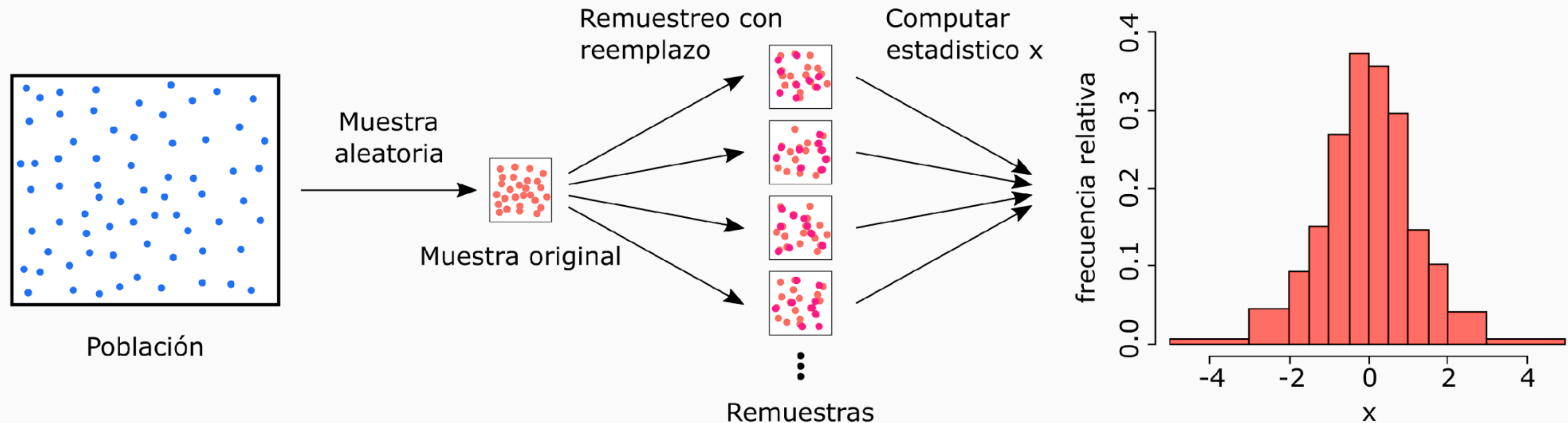
- Una forma de obtener diversidad en el ensamble es utilizando **modelos diferentes** (arquitecturas diferentes para cada predictor).
- Otra alternativa es usar la misma arquitectura, pero entrenándola con **distintos subconjuntos del set de datos**.

¿Si elegimos la segunda estrategia (mismo modelo, diferentes subconjuntos de datos), distinguimos dos métodos de construcción de esos subconjuntos:

- **Bagging**: muestreo **con reposición** (por lo que una misma observación puede aparecer varias veces en el entrenamiento de un mismo predictor).
- **Pasting** : muestreo **sin reposición**.

BAGGING

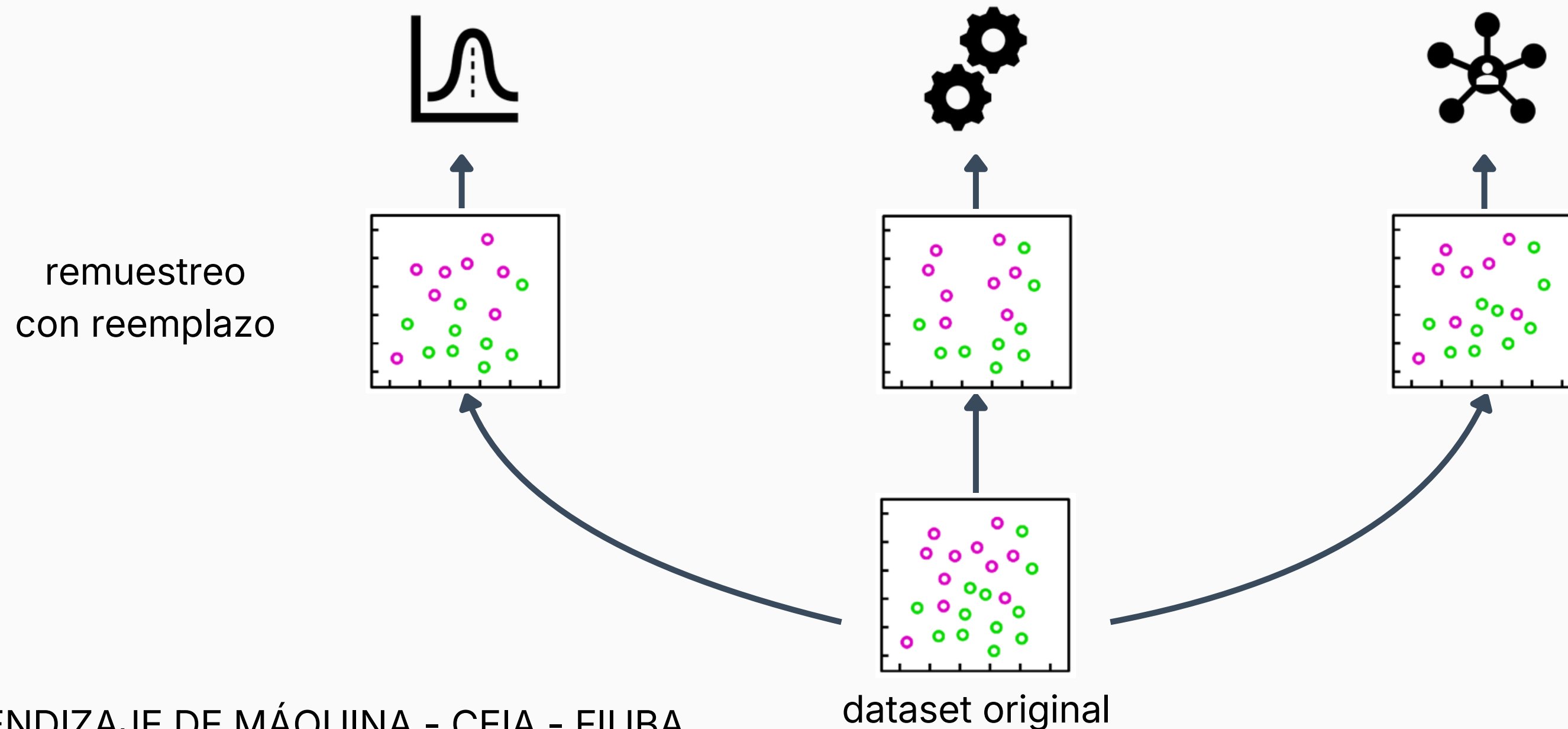
En estadística, la técnica de re-muestreo con reposición se denomina **Bootstrapping**.



BAGGING

Para realizar **Bagging** (*bootstrap aggregating*), utilizamos modelos que tengan mucha varianza y poco sesgo.

Cada uno de los modelos del ensamble puede ser entrenado en paralelo, lo que permite escalar la solución.



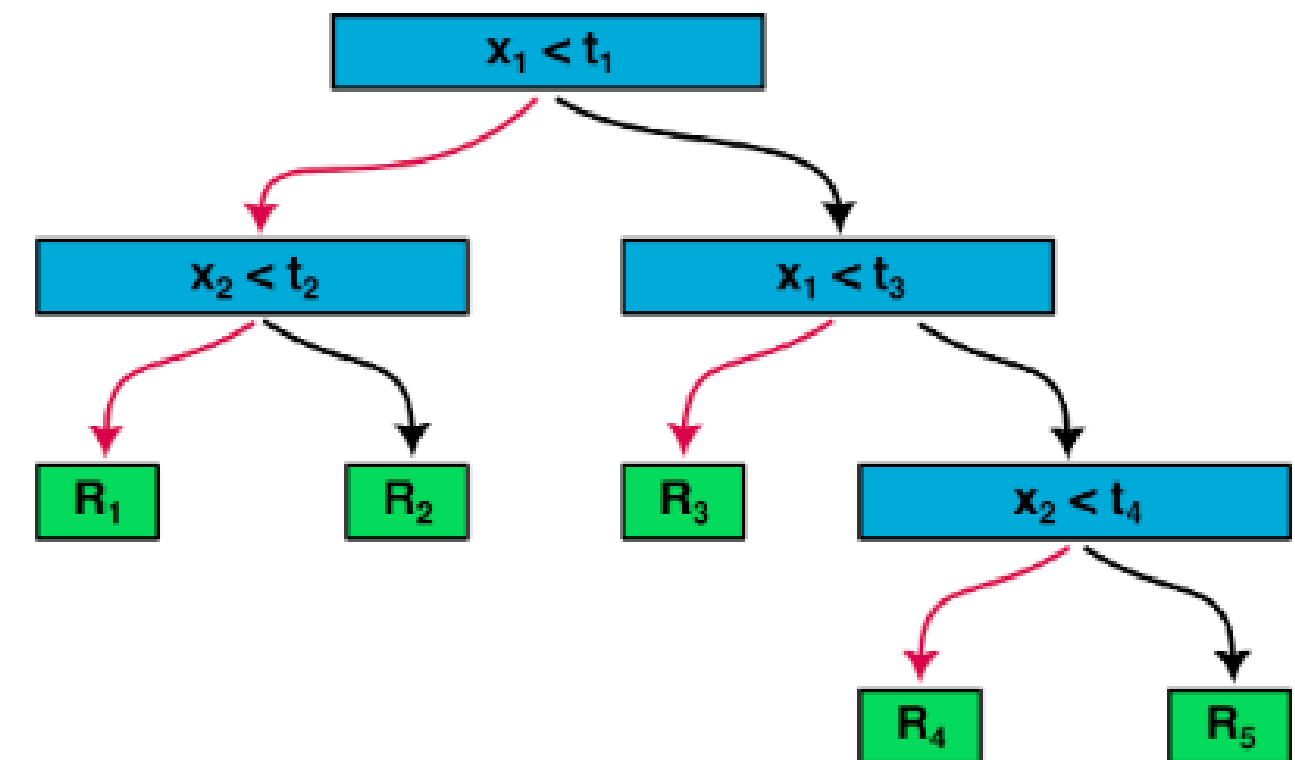
BAGGING

Lo más común es utilizar un solo tipo de modelo base, normalmente, **árboles de decisión**.

Cada árbol se entrena con una muestra bootstrap diferente del conjunto de entrenamiento, y produce una predicción individual, diferente.

Luego el ensamble combina esas predicciones:

- Clasificación:
 - votación mayoritaria.
- Regresión:
 - promedio de las predicciones.



BAGGING

¿Por qué funciona?

- Generalmente, se usan **árboles profundos**, con bajo sesgo y alta varianza.
- Al combinar muchos árboles, los errores individuales tienden a compensarse.
- El promedio (o la votación) reduce la varianza sin aumentar significativamente el sesgo.
- Bagging mejora sustancialmente los resultados cuando se escala a cientos o miles de árboles.

***BOSQUES
ALEATORIOS
(Random Forests)***

BOSQUES ALEATORIOS

Los **bosques aleatorios** (*Random Forests*) son una mejora respecto al uso exclusivo de **Bagging**.

El funcionamiento es similar, pero agregan un paso importante:

- En cada división se considera solo una **selección aleatoria de atributos**. Por regla general, la cantidad de atributos a utilizar (m) es aproximadamente la raíz cuadrada del total de atributos disponibles (p).

Ejemplo: Si tenemos $p=13$ atributos, usaremos $m \approx 4$ (ya que $\sqrt{13} \approx 3.6$). En cada nodo del árbol, el algoritmo elegirá una combinación diferente de 4 atributos al azar para decidir la mejor división.

BOSQUES ALEATORIOS

El razonamiento

Supongamos que existe un atributo que es un **predictor muy fuerte**, acompañado de otros moderadamente fuertes.

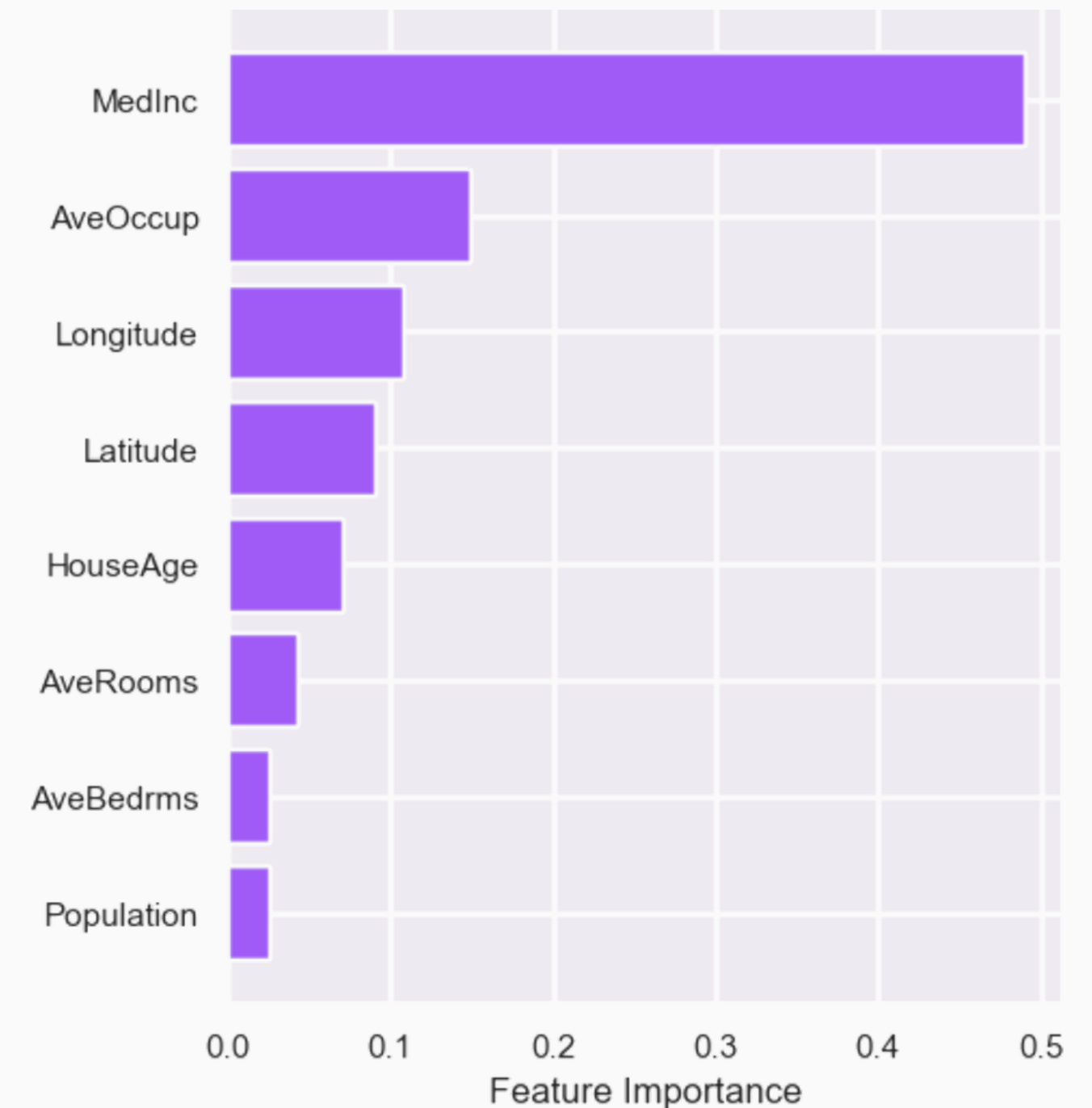
Si aplicamos **Bagging**, la gran mayoría de los árboles tenderán a utilizar ese atributo fuerte en la bifurcación inicial (nodo raíz). Por consiguiente, los árboles resultantes serán muy similares entre sí, generando una alta correlación y **limitando la capacidad del ensamble para reducir la varianza**.

Random Forests soluciona esto al restringir los atributos disponibles en cada nodo. En promedio, una fracción de $(p-m)/p$ de las divisiones no tendrá en cuenta ese atributo fuerte, lo que obliga a los árboles a explorar otras variables y se logra descorrelacionar las predicciones.

BOSQUES ALEATORIOS

El poder de los Bosques Aleatorios se hace evidente al crear modelos para datos con **muchos atributos**.

Estos modelos tienen la capacidad de determinar automáticamente qué variables son importantes y descubrir relaciones complejas entre ellas (interacciones), que a menudo pasan desapercibidas para otros algoritmos.



BOSQUES ALEATORIOS

Existen dos enfoques principales:

- **Por disminución de la precisión (Permutación)**. Se mide cuánto cae la precisión del modelo si los valores de un atributo se permutan aleatoriamente. *Al mezclar los valores de una columna, rompemos su relación con la variable objetivo, eliminando su poder predictivo. Si el error aumenta mucho, el atributo era importante.*
- **Por disminución de la impureza (Impureza media)**. Se calcula la disminución media de impureza (Gini o Entropía) producida por un atributo en los nodos donde se utiliza, para ver en qué medida la variable logra separar los datos. *Cuanto mayor sea esta reducción, mayor es la importancia de la variable.*

BOOSTING

BOOSTING

Este tipo de arquitecturas consiste en ensambles que, al igual que los vistos anteriormente, combinan varios predictores para lograr una mejor predicción. A diferencia de Bagging, el entrenamiento en Boosting es **secuencial**.

Lo más común es utilizar árboles de decisión con pocos nodos (poco profundos). Este es un proceso de aprendizaje lento, ya que cada paso depende de los árboles anteriores. Sin embargo, estadísticamente, un proceso de aprendizaje lento suele conducir a mejores resultados (o un mejor modelo final).

BOOSTING. Adaptive Boosting (AdaBoost)

Existen varios algoritmos de Boosting con distintos niveles de complejidad, pero el más sencillo de entender es **AdaBoost**. Su proceso de entrenamiento simplificado es:

1. **Inicialización.** Se comienza asignando **pesos iguales** a todas las muestras del conjunto de entrenamiento.
2. **Entrenamiento.** Se entrena un modelo base (*weak learner*) utilizando los datos ponderados.
3. **Evaluación.** Se calcula el error del modelo. Las observaciones mal clasificadas adquieren mayor importancia para el siguiente paso.
4. **Peso del modelo (α).** Se asigna un peso al modelo en función de su precisión. Los modelos más acertados tienen mayor influencia en la votación final.
5. **Actualización de pesos.** Se incrementan los pesos de las instancias mal clasificadas para que el próximo modelo se vea "forzado" a enfocarse en ellas.
6. **Iteración.** Se repiten los pasos 2 a 5 varias veces (generalmente de 50 a 100 iteraciones).

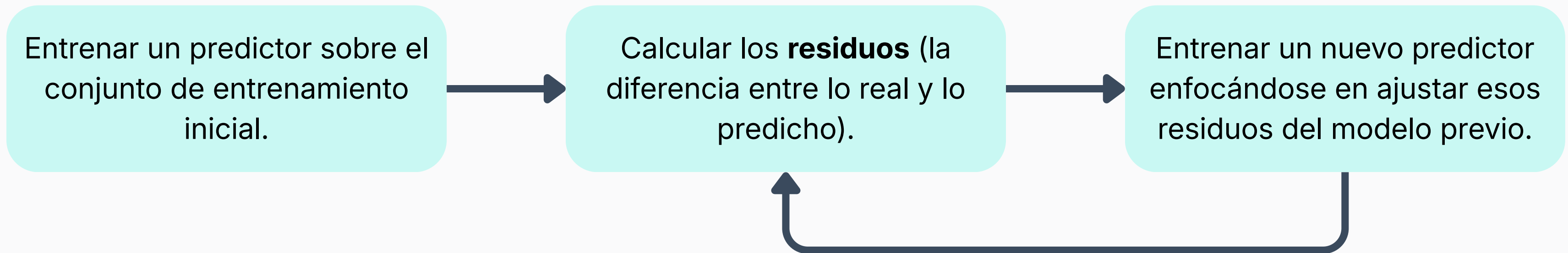
Predicción final: se combinan las predicciones de todos los modelos débiles, ponderadas por sus respectivos pesos (α) mediante la fórmula:

$$\hat{F}(X) = \sum_{m=1}^M \alpha_m \hat{f}_m(X)$$

BOOSTING. Gradient Boosting.

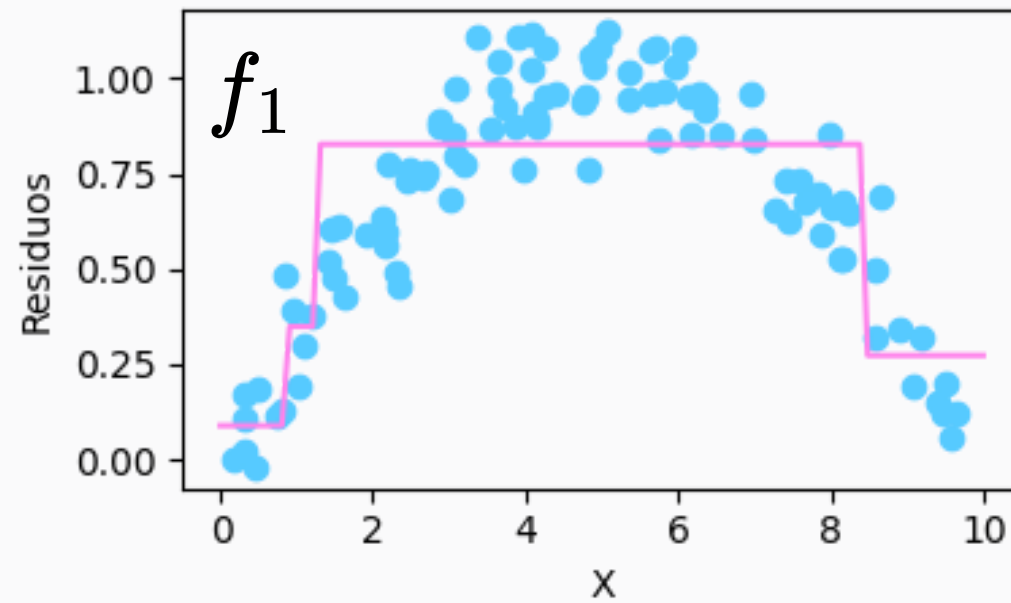
Mientras que **AdaBoost** pondera las observaciones con una función exponencial, **Gradient Boosting** utiliza el gradiente de la función de costo. Con este gradiente se ajustan los residuos y se ponderan los nuevos modelos (*weak learners*).

Cada nuevo predictor que se agrega al ensamble intenta corregir los errores de su predecesor.

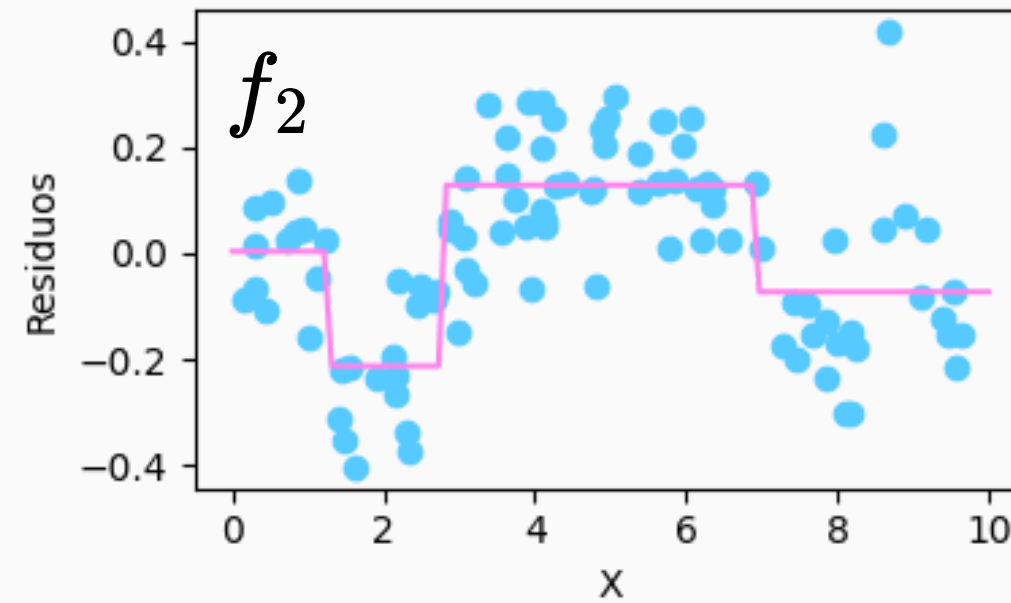


BOOSTING. Gradient Boosting.

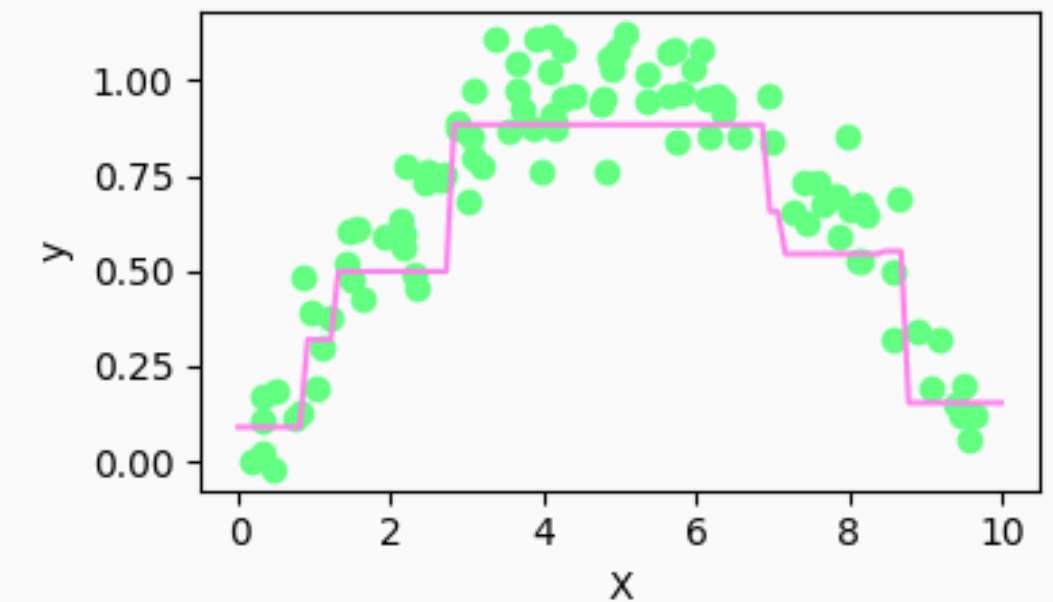
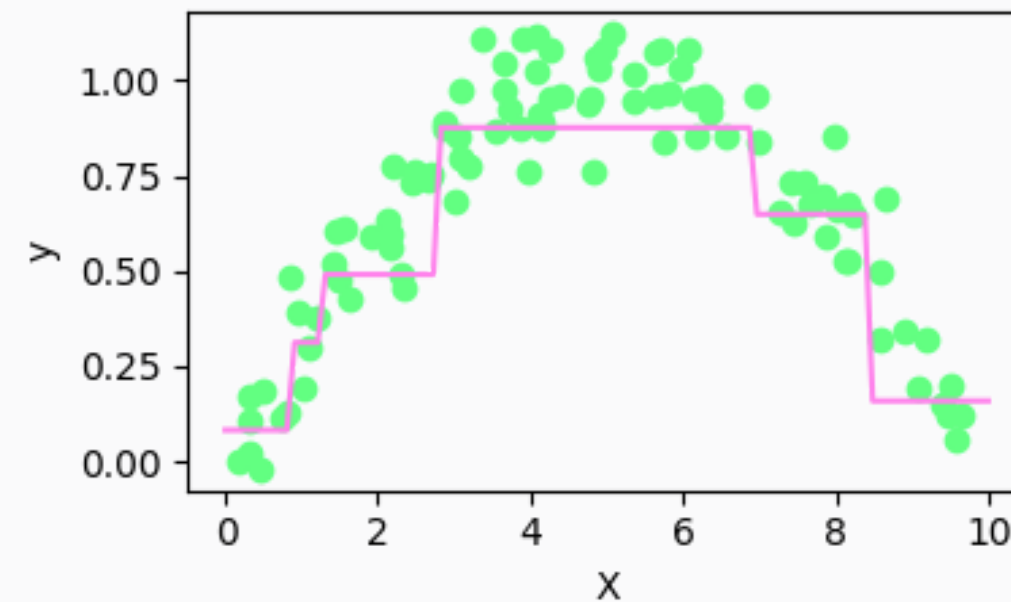
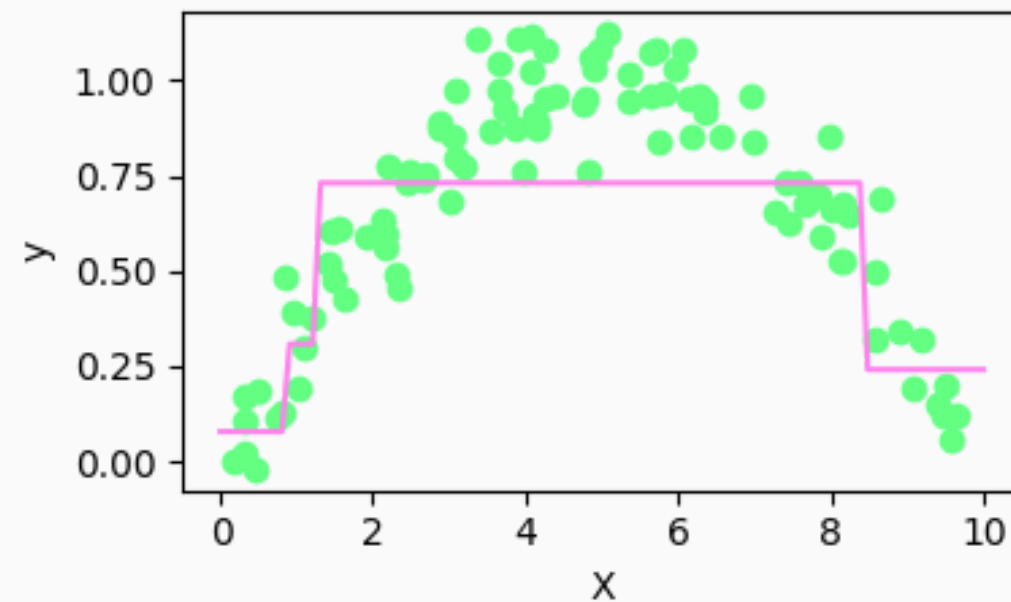
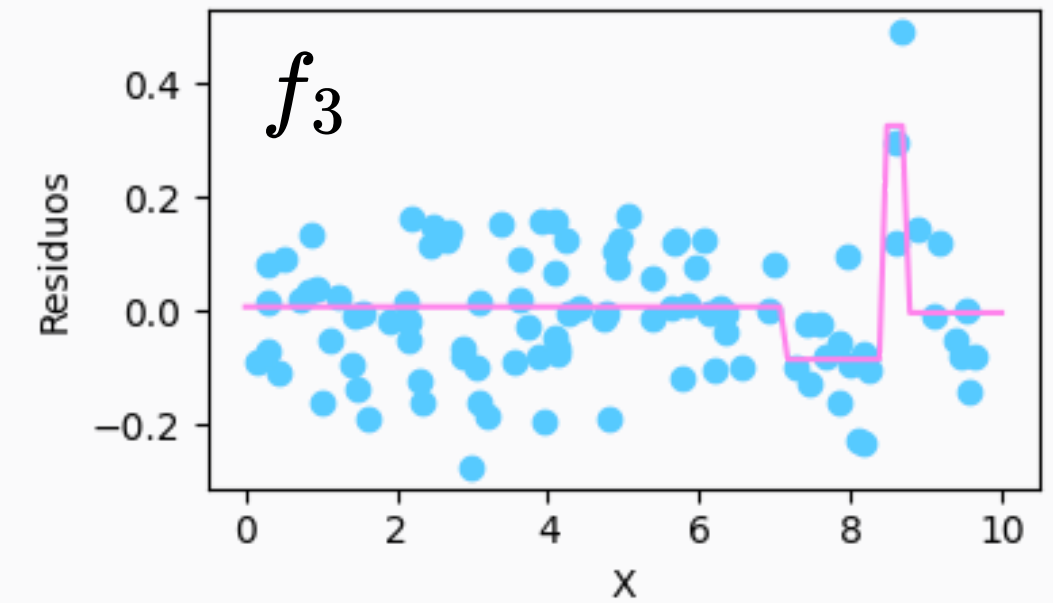
Iteración 1



Iteración 2



Iteración 3



Salida de ensamble: $F = \alpha_1 f_1$

$$F = \alpha_1 f_1 + \alpha_2 f_2$$

$$F = \alpha_1 f_1 + \alpha_2 f_2 + \alpha_3 f_3$$

BAGGING vs BOOSTING

	Bagging	Boosting
<i>Método de construcción del conjunto de modelos</i>	Múltiples modelos de aprendizaje automático independientes entrenados en subconjuntos aleatorios de datos de entrenamiento	Construye modelos secuencialmente , centrándose en corregir los errores del modelo anterior
<i>Tipo de modelos</i>	Adecuado para cualquier modelo de aprendizaje automático	Se enfoca en modelos débiles y mejora su precisión al combinar modelos débiles
<i>Proceso de predicción</i>	Promedio de las predicciones de todos los modelos del conjunto	Asigna diferentes pesos a cada modelo y los combina para hacer una predicción final
<i>Sensibilidad al ruido</i>	Menos sensible al ruido y a los valores atípicos	Puede ser más sensible al ruido y a los valores atípicos
<i>Tiempo de entrenamiento</i>	Puede ser más rápido ya que los modelos se construyen de forma independiente	Puede requerir más tiempo de entrenamiento ya que cada modelo se entrena secuencialmente
<i>Ventajas</i>	Reduce la varianza, mejora la precisión y puede manejar grandes conjuntos de datos	Mejora la precisión y el rendimiento del modelo mediante la corrección de errores y la creación de modelos más precisos
<i>Desventajas</i>	Puede aumentar el sesgo y no mejorar la precisión si los modelos del conjunto son similares	Puede ser más propenso al sobreajuste y al ruido si se usa un modelo débil o ruidoso

XGBOOST

XGBOOST

Una de las librerías más utilizada en Boosting es XGBoost, una implementación de Gradient Boosting estocástico desarrollada por Tianqi Chen y Carlos Guestrin en la Universidad de Washington.



[XGBoost: A Scalable Tree Boosting System](#)

XGBOOST

El modelo **XGBoost** cuenta con una gran cantidad de hiperparámetros. Dos de los más importantes son:

- **subsample**: controla la fracción de observaciones que se muestrean al azar en cada iteración. Al usar una submuestra, el algoritmo incorpora aleatoriedad (similar a un *Random Forest*) aunque aquí el muestreo se realiza típicamente sin reemplazo (**pasting**).
- **eta**: es un factor de contracción aplicado al peso α_m de cada nuevo árbol que se va agregando. Su función es escalar (reducir) la contribución de cada árbol nuevo al modelo total. Este parámetro es fundamental para controlar la velocidad de aprendizaje y evitar el sobreajuste (overfitting).

XGBOOST

En Python, **XGBoost** ofrece dos interfaces distintas:

- **Nativa**: más cercana al paradigma funcional y similar a la implementación en R.
- **Scikit-learn API**: diseñada para imitar la estructura de scikit-learn. Para mantener consistencia con esta librería, algunos parámetros cambian de nombre (ejemplo: `eta` → `learning_rate`).

XGBOOST

Dada su tendencia al overfitting, XGBoost incorpora potentes mecanismos de **regularización**.

Para controlar la complejidad, dispone de dos parámetros:

- *reg_alpha*: corresponde a la regularización L1 (tipo Lasso).
- *reg_lambda*: Corresponde a la regularización L2 (tipo Ridge).

Incrementar estos valores penaliza la complejidad del modelo, lo que ayuda a reducir el tamaño de los pesos en las hojas y favorece árboles más conservadores.

XGBOOST

Otras implementaciones muy populares de los algoritmos de Gradient Boosting son **LightGBM** y **CatBoost**, ambos basados en árboles.





Vamos a practicar un
poco ...

En la próxima clase veremos ...

Aprendizaje No Supervisado. Clustering. K-Means. Hierarchical clustering. Modelo de Mixtura Gaussiana. Suma de cuadrados intracluster. Índice de la silueta. Fuerza de predicción.